

Unit: 6 Device Management

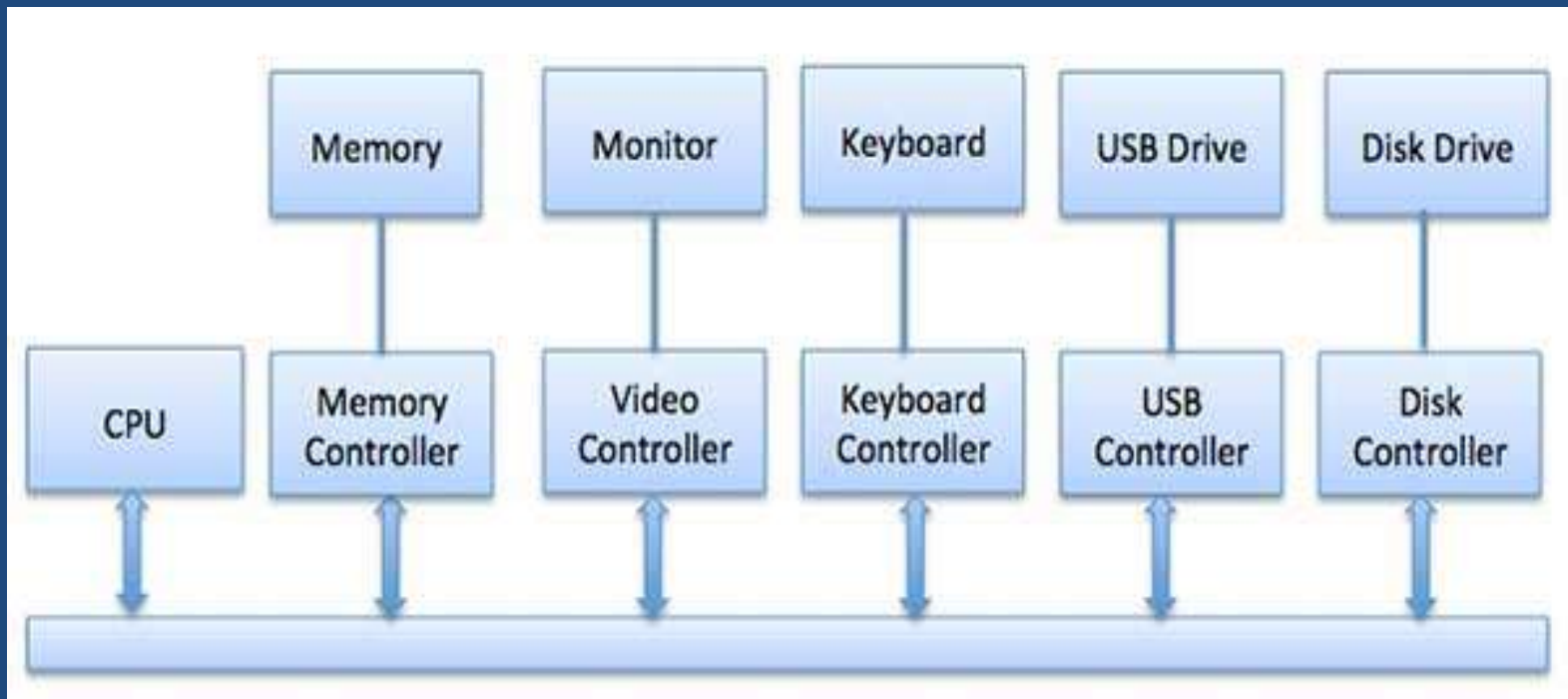
Er. Binod Kumar Rajbhar

6.1 Classification of IO devices

- ❖ One of the important jobs of an Operating System is to manage various I/O devices including mouse, keyboards, touch pad, disk drives, display adapters, USB devices, Bit-mapped screen, LED, Analog-to-digital converter, On/off switch, network connections, audio I/O, printers etc.
- ❖ An I/O system is required to take an application I/O request and send it to the physical device, then take whatever response comes back from the device and send it to the application.
- ❖ **I/O devices can be divided into two categories –**
 - **Block devices** – A block device is one with which the driver communicates by sending entire blocks of data. For example, Hard disks, USB cameras, Disk-On-Key etc.
 - **Character devices** – A character device is one with which the driver communicates by sending and receiving single characters (bytes, octets). For example, serial ports, parallel ports, sounds cards etc.

Device Controllers:

- ❖ Device drivers are software modules that can be plugged into an OS to handle a particular device.
- ❖ Operating System takes help from device drivers to handle all I/O devices.
- ❖ The Device Controller works like an interface between a device and a device driver.
- ❖ I/O units (Keyboard, mouse, printer, etc.) typically consist of a mechanical component and an electronic component where electronic component is called the device controller.
- ❖ There is always a device controller and a device driver for each device to communicate with the Operating Systems.
- ❖ A device controller may be able to handle multiple devices.
- ❖ As an interface its main task is to convert serial bit stream to block of bytes, perform error correction as necessary.
- ❖ Any device connected to the computer is connected by a plug and socket, and the socket is connected to a device controller.
- ❖ Following is a model for connecting the CPU, memory, controllers, and I/O devices where CPU and device controllers all use a common bus for communication.



Memory mapped IO vs. IO mapped IO

MEMORY MAPPED IO VERSUS IO MAPPED IO	
MEMORY MAPPED IO	IO MAPPED IO
A method to perform input/output (I/O) operations between the central processing unit (CPU) and peripheral devices in a computer that uses one address space for memory and IO devices	A method to perform input/output (I/O) operations between the central processing unit (CPU) and peripheral devices in a computer that uses two separate address spaces for memory and IO device
Uses the same address space for both memory and IO devices	Uses two separate address spaces for memory and IO device
As the memory mapped IO uses one address space for both IO and memory, the available addresses for memory are minimum	All the addresses can be used by the memory
Uses the same instructions for both IO and memory operations	Uses separate instructions for read and write operations in IO and memory
Less efficient	More efficient
	Visit www.PEDIAA.com

Differentiate between I/O mapped I/O and Memory mapped I/O of 8086.

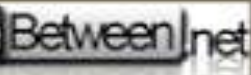
Sr. No.	I/O Mapped I/O	Memory Mapped I/O
1.	I/O device is treated as an I/O device and hence given an I/O address.	I/O device is treated like a memory device and hence given a memory address.
2.	I/O device has an 8 or 16 bit I/O address.	I/O device has a 20 bit Memory address.
3.	I/O device is given IOR# and IOW# control signals	I/O device is given MEMR# and MEMW# control signals
4.	Decoding is easier due to lesser address lines	Decoding is more complex due to more address lines
5.	Decoding is cheaper	Decoding is more expensive
6.	Works faster due to less delays	More gates add more delays hence slower
7.	Allows max $2^{16} = 65536$ I/O devices	Allows many more I/O devices as I/O addresses are now 20 bits.
8.	I/O devices can only be accessed by IN and OUT instructions.	I/O devices can now be accessed using any memory instruction.
9.	ONLY AL/ AH/ AX registers can be used to transfer data with the I/O device	Any register can be used to transfer data with the I/O device

Interrupt IO vs. Polled IO

<u>Basis for Comparison</u>	<u>Interrupt</u>	<u>Polling</u>
Basic	Device notifies CPU that it needs CPU attention.	CPU constantly checks device status whether it needs CPU's attention.
Mechanism	An interrupt is a hardware mechanism.	Polling is a Protocol.
Servicing	Interrupt handler services the Device.	CPU services the device.
Indication	Interrupt-request line indicates that device needs servicing.	Command-ready bit indicates the device needs servicing.
CPU	CPU is disturbed only when a device needs servicing, which saves CPU cycles.	CPU has to wait and check whether a device needs servicing which wastes lots of CPU cycles.
Occurrence	An interrupt can occur at any time.	CPU polls the devices at regular interval.
Efficiency	Interrupt becomes inefficient when devices keep on interrupting the CPU repeatedly.	Polling becomes inefficient when CPU rarely finds a device ready for service.
Example	Let the bell ring then open the door to check who has come.	Constantly keep on opening the door to check whether anybody has come.

POLLING VERSUS INTERRUPT

Criterion	Polling	Interrupt
Background	Checking at regular intervals	Processor is called if needed
Mechanism	Protocol	Mechanism
Servicing	CPU	Interrupt handler
CPU	On hold	Called if needed
Appearance	On regular interval	Anytime
Advantages	Simple program, transmission reliability, no need for additional chips	Serves multiple devices, flexible, efficient
Disadvantages	Standby time, time waste	More complex, time consuming

Difference Between 

INTERRUPT VERSUS POLLING

INTERRUPT

An event that is triggered by external components other than the CPU that alerts the CPU to perform a certain action

When an interrupt occurred, the interrupt handler executes

Can occur at any time

Interrupt-request line indicates that device needs a service

Does not waste much CPU cycles

It is inefficient when the device interrupts the CPU frequently

POLLING

An activity of sampling the status of an external device by a client program as a synchronous activity

In polling, the CPU provide the service

Occurs at regular intervals

Command ready bit indicates the device needs a service

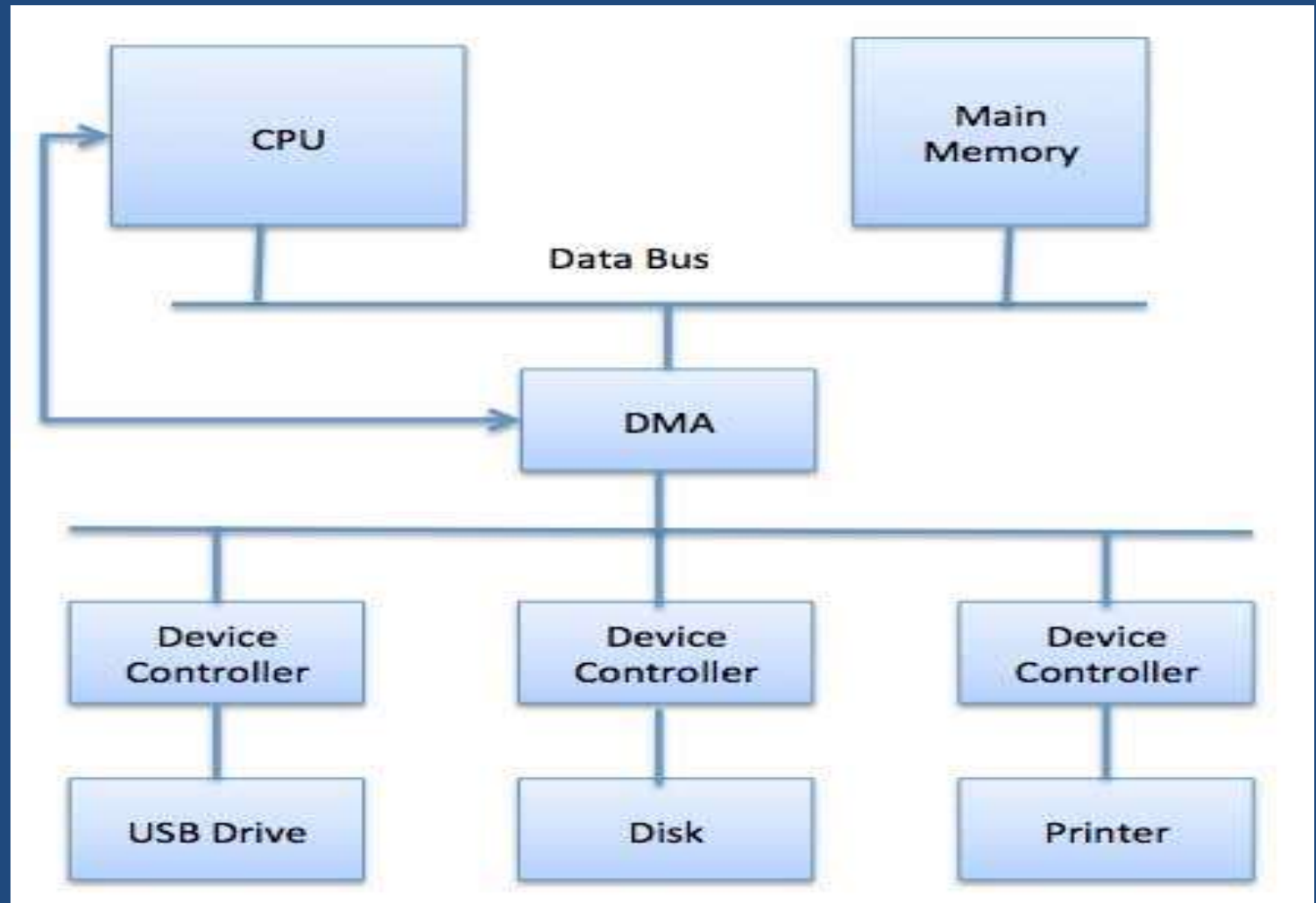
Wastes lot of CPU cycles

It is inefficient in polling, when the CPU rarely finds requests from the devices

Visit www.PEDIAA.com

Direct Memory Access (DMA):

- ❖ Direct Memory Access (DMA) means CPU grants I/O module authority to read from or write to memory without involvement.
- ❖ DMA module itself controls exchange of data between main memory and the I/O device.
- ❖ CPU is only involved at the beginning and end of the transfer and interrupted only after entire block has been transferred.
- ❖ Direct Memory Access needs a special hardware called DMA controller (DMAC) that manages the data transfers and arbitrates access to the system bus.
- ❖ The controllers are programmed with source and destination pointers (where to read/write the data), counters to track the number of transferred bytes, and settings, which includes I/O and memory types, interrupts and states for the CPU cycles.

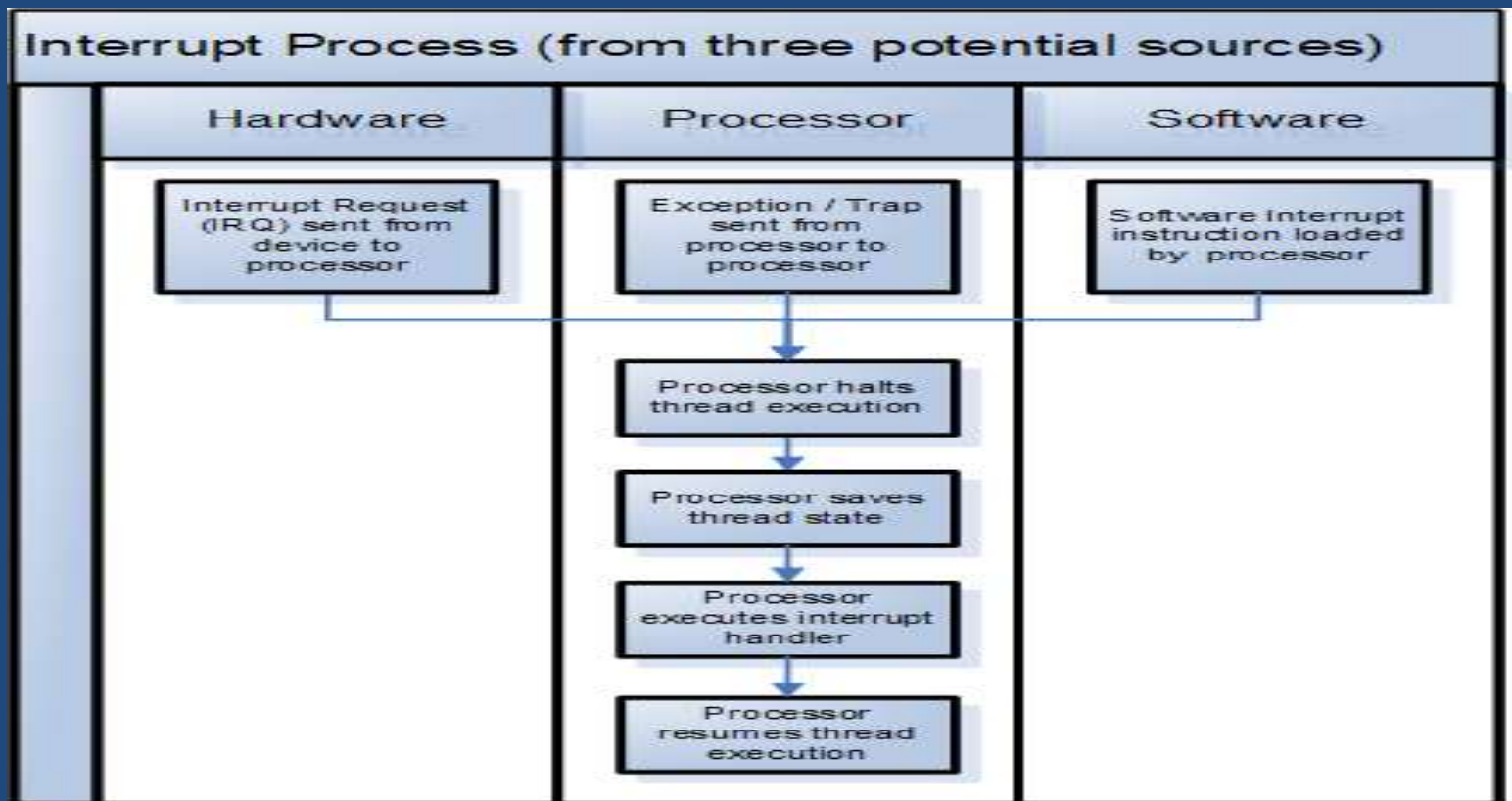


❖ The operating system uses the DMA hardware as follows –

Step	Description
1	Device driver is instructed to transfer disk data to a buffer address X.
2	Device driver then instruct disk controller to transfer data to buffer.
3	Disk controller starts DMA transfer.
4	Disk controller sends each byte to DMA controller.
5	DMA controller transfers bytes to buffer, increases the memory address, decreases the counter C until C becomes zero.
6	When C becomes zero, DMA interrupts CPU to signal transfer completion.

6.2 Goals of IO Software

Principles of I/O Software:



Goals of the I/O Software

❖ Device independence:

- ✓ A key concept in the design of I/O software is known as **device independence**.
- ✓ It means that I/O devices should be accessible to programs without specifying the device in advance.

❖ Uniform Naming,

- ✓ simply be a string or an integer and not depend on the device in any way.
- ✓ In UNIX, all disks can be integrated in the file-system hierarchy in arbitrary ways so the user need not be aware of which name corresponds to which device.

❖ Error Handling:

- ✓ If the controller discovers a read error, it should try to correct the error itself if it can.
- ✓ If it cannot, then the device driver should handle it, perhaps by just trying to read the block again.
- ✓ In many cases, error recovery can be done transparently at a low level without the upper levels even knowing about the error.

❖ **Synchronous (blocking) and Asynchronous (interrupt-driven) transfers:**

- ✓ Most physical I/O is asynchronous, however, some very high-performance applications need to control all the details of the I/O, so some operating systems make asynchronous I/O available to them.

❖ **Buffering:** Often data that come off a device cannot be stored directly in their final destination.

❖ **Sharable and Dedicated devices:**

- ✓ Some I/O devices, such as disks, can be used by many users at the same time.
- ✓ No problems are caused by multiple users having open files on the same disk at the same time.
- ✓ Other devices, such as printers, have to be dedicated to a single user until that user is finished.
- ✓ Then another user can have the printer.
- ✓ Introducing dedicated (unshared) devices also introduces a variety of problems, such as deadlocks.
- ✓ Again, the operating system must be able to handle both shared and dedicated devices in a way that avoids problems.

Programmed I/O

- ❖ This is one of the three fundamentally different ways that I/O can be performed.
- ❖ The programmed I/O was the most simple type of I/O technique for the exchanges of data or any types of communication between the processor and the external devices.
- ❖ With programmed I/O, data are exchanged between the processor and the I/O module.
- ❖ The processor executes a program that gives it direct control of the I/O operation, including sensing device status, sending a read or write command, and transferring the data.
- ❖ When the processor issues a command to the I/O module, it must wait until the I/O operation is complete.
- ❖ If the processor is faster than the I/O module, this is wasteful of processor time.
- ❖ The overall operation of the programmed I/O can be summaries as follow:
 - The processor is executing a program and encounters an instruction relating to I/O operation.
 - The processor then executes that instruction by issuing a command to the appropriate I/O module.
 - The I/O module will perform the requested action based on the I/O command issued by the processor (READ/WRITE) and set the appropriate bits in the I/O status register.
 - The processor will periodically check the status of the I/O module until it find that the operation is complete.

- **Programmed I/O Commands**

- ❖ To execute an I/O-related instruction, the processor issues an address, specifying the particular I/O module and external device, and an I/O command.
- ❖ There are four types of I/O commands that an I/O module may receive when it is addressed by a processor:
 - . **Control**
 - . **Test**
 - . **Read**
 - . **Write**

Advantages of Programmed I/O

- . Simple to implement
- . Very little hardware support

Disadvantages of Programmed I/O

- . Busy waiting
- . Ties up CPU for long period with no useful work

Interrupt-Driven I/O:

- ❖ Interrupt driven I/O is an alternative scheme dealing with I/O.
- ❖ Interrupt I/O is a way of controlling input/output activity whereby a peripheral or terminal that needs to make or receive a data transfer sends a signal.
- ❖ This will cause a program interrupt to be set.
- ❖ At a time appropriate to the priority level of the I/O interrupt.
- ❖ Relative to the total interrupt system, the processors enter an interrupt service routine.

Interrupt I/O Inputs

- ✓ For input, the device interrupts the CPU when new data has arrived and is ready to be retrieved by the system processor.
- ✓ The actual actions to perform depend on whether the device uses I/O ports or memory mapping.

Interrupt I/O Outputs

- ✓ For output, the device delivers an interrupt either when it is ready to accept new data or to acknowledge a successful data transfer.
- ✓ Memory-mapped and DMA-capable devices usually generate interrupts to tell the system they are done with the buffer.

Operations in Interrupt I/O :

- . CPU issues read command.
- . I/O module gets data from peripheral whilst CPU does other work.
- . I/O module interrupts CPU.
- . CPU requests data.
- . I/O module transfers data.

Advantages of Interrupt-Driven I/O:

- Its faster than Programmed I/O.
- Efficient too.

Disadvantages of Interrupt-Driven I/O ;

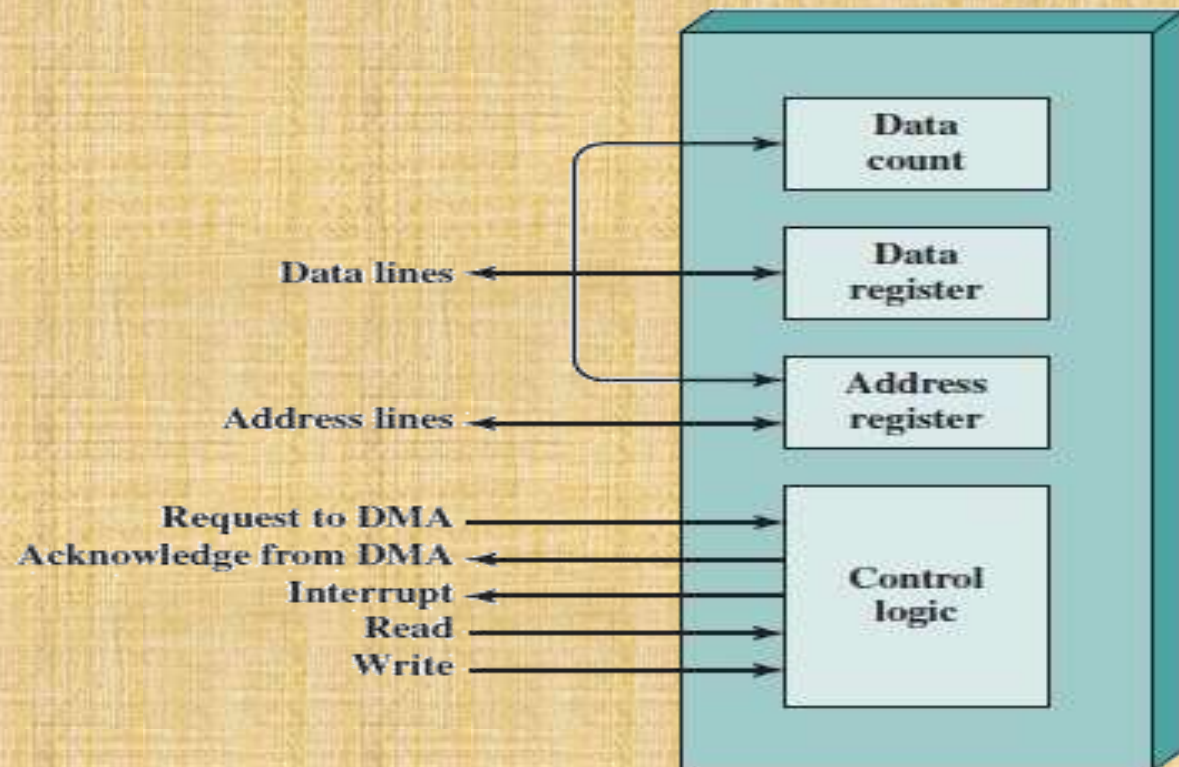
- It can be tricky to write if using a low level language.
- It can be tough to get various pieces to work well together.

I/O Using DMA:

- ❖ Direct Memory Access is a technique for transferring data within main memory and external device without passing it through the CPU.
- ❖ DMA is a way to improve processor activity and I/O transfer rate by taking-over the job of transferring data from processor, and letting the processor to do other tasks.
- ❖ This technique overcomes the drawbacks of other two I/O techniques which are the time consuming process when issuing a command for data transfer and tie-up the processor in data transfer while the data processing is neglected.
- ❖ It is more efficient to use DMA method when large volume of data has to be transferred.
- ❖ For DMA to be implemented, processor has to share its' system bus with the DMA module.
- Therefore, the DMA module must use the bus only when the processor does not need it, or it must force the processor to suspend operation temporarily

Operations of Direct Memory Access:

- Read or Write Command
- Control Lines Communication
- Data Lines Communication



```
copy_from_user(buffer, p, count);  
set_up_DMA_controller();  
scheduler();
```

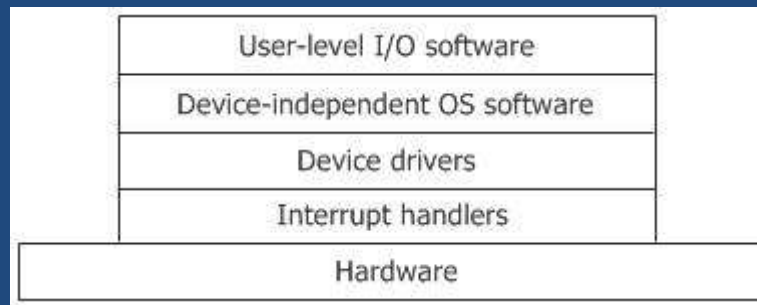
```
acknowledge_interrupt();  
unblock_user();  
return_from_interrupt();
```

Advantages of DMA :

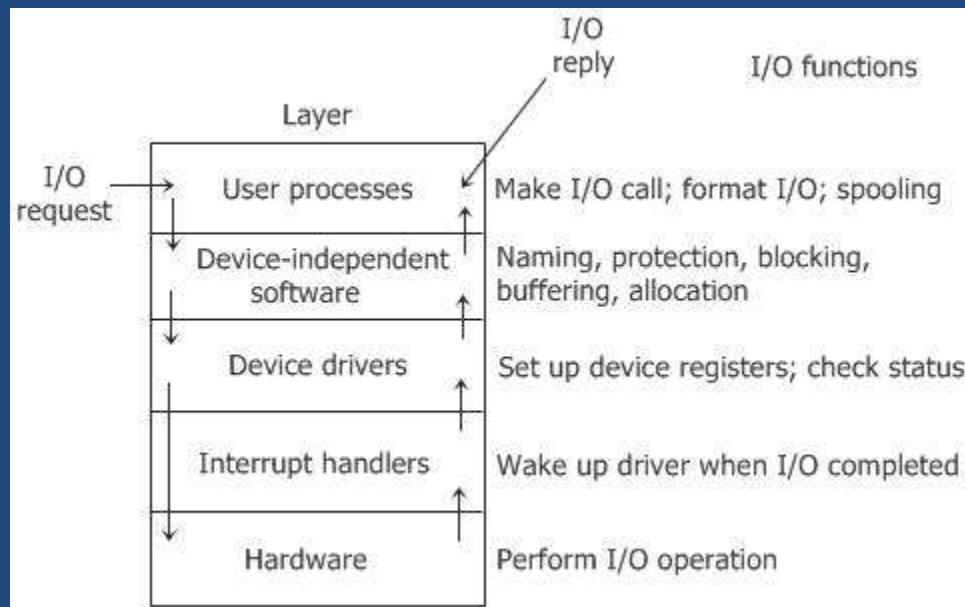
- Speed: no waiting due to much shorter execution path and no rotation delay.
- **OS Input/Output Software Layers**

IO Software Layers:

- Basically, input/output software organized in the following four layers:
 - Interrupt handlers
 - Device drivers
 - Device-independent input/output software
 - User-space input/output software
- In every input/output software, each of the above given four layer has a well-defined function to perform and a well-defined interface to the adjacent layers.
- The figure given below shows all the layers along with hardware of the input/output software system.



Here is another figure shows all the layers of the input/output software system along with their principal functions.



❖ Interrupt Handlers:

- Whenever the interrupt occurs, then the interrupt procedure does whatever it has to in order to handle the interrupt.

❖ Device Drivers:

- Basically, device drivers is a device-specific code just for controlling the input/output device that are attached to the computer system.

❖ Device-Independent Input/output Software

- ✓ In some of the input/output software is device specific and other parts of that input/output software are device-independent.
- ✓ The exact boundary between the device-independent software and drivers is device dependent, just because of that some functions that could be done in a device-independent way sometime be done in the drivers, for efficiency or any other reasons.
- ✓ Here are the list of some functions that are done in the device-independent software:
 - Uniform interfacing for device drivers
 - Buffering
 - Error reporting
 - Allocating and releasing dedicated devices
 - Providing a device-independent block size

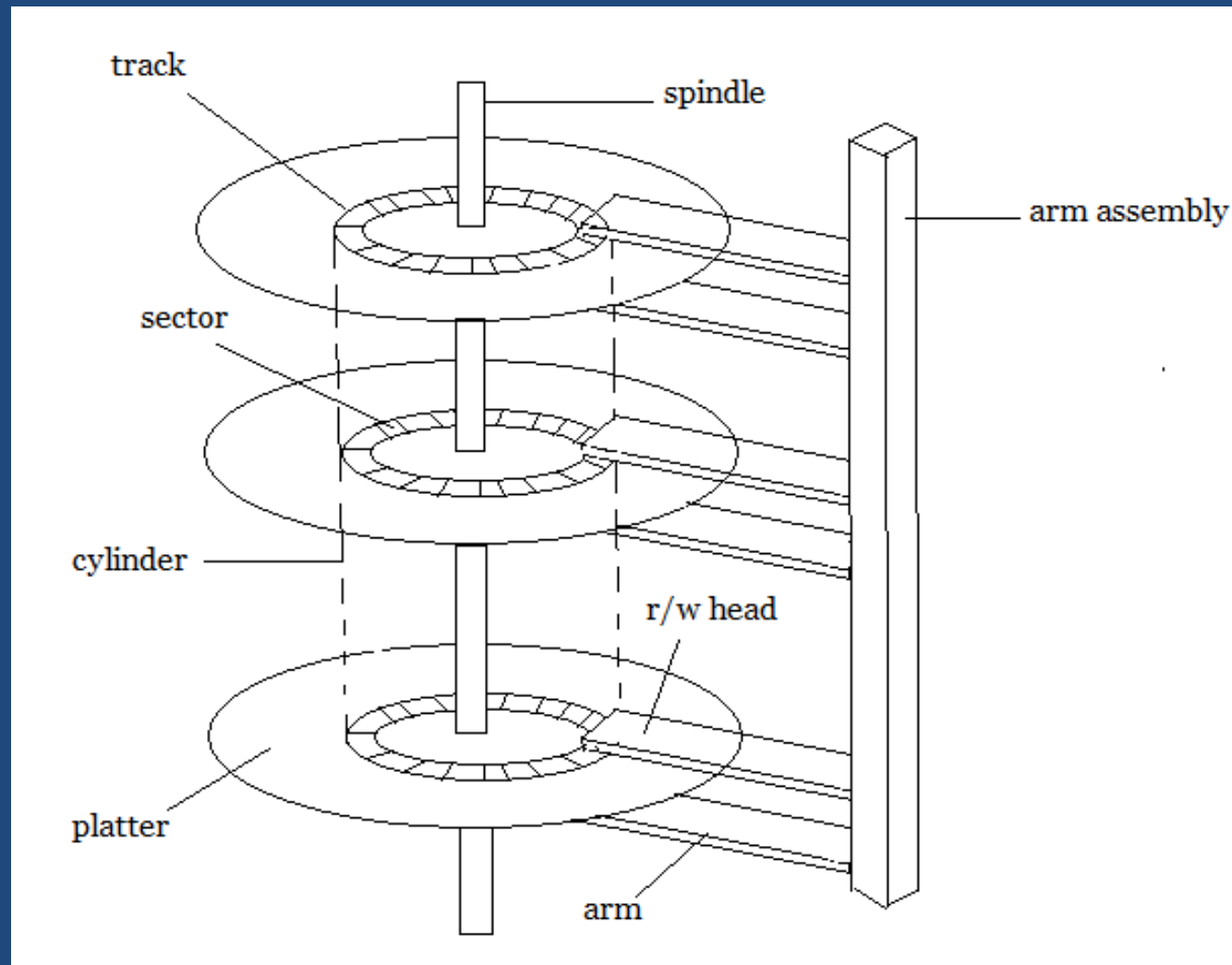
❖ User-Space Input/output Software:

- ✓ Generally most of the input/output software is within the operating system (OS), and some small part of that input/output software consists of libraries that are linked with the user programs and even whole programs running outside the kernel.

6.3 Disk Structure

- Magnetic Disk Structure

- ❖ In modern computers, most of the secondary storage is in the form of magnetic disks.
- ❖ Hence, knowing the structure of a magnetic disk is necessary to understand how the data in the disk is accessed by the computer.



- **Structure of a magnetic disk**

- ❖ A magnetic disk contains several **platters**.
- ❖ Each platter is divided into circular shaped **tracks**.
- ❖ The length of the tracks near the center is less than the length of the tracks farther from the center.
- ❖ Each track is further divided into **sectors**, as shown in the figure.
- ❖ Tracks of the same distance from center form a cylinder.
- ❖ A read-write head is used to read data from a sector of the magnetic disk.

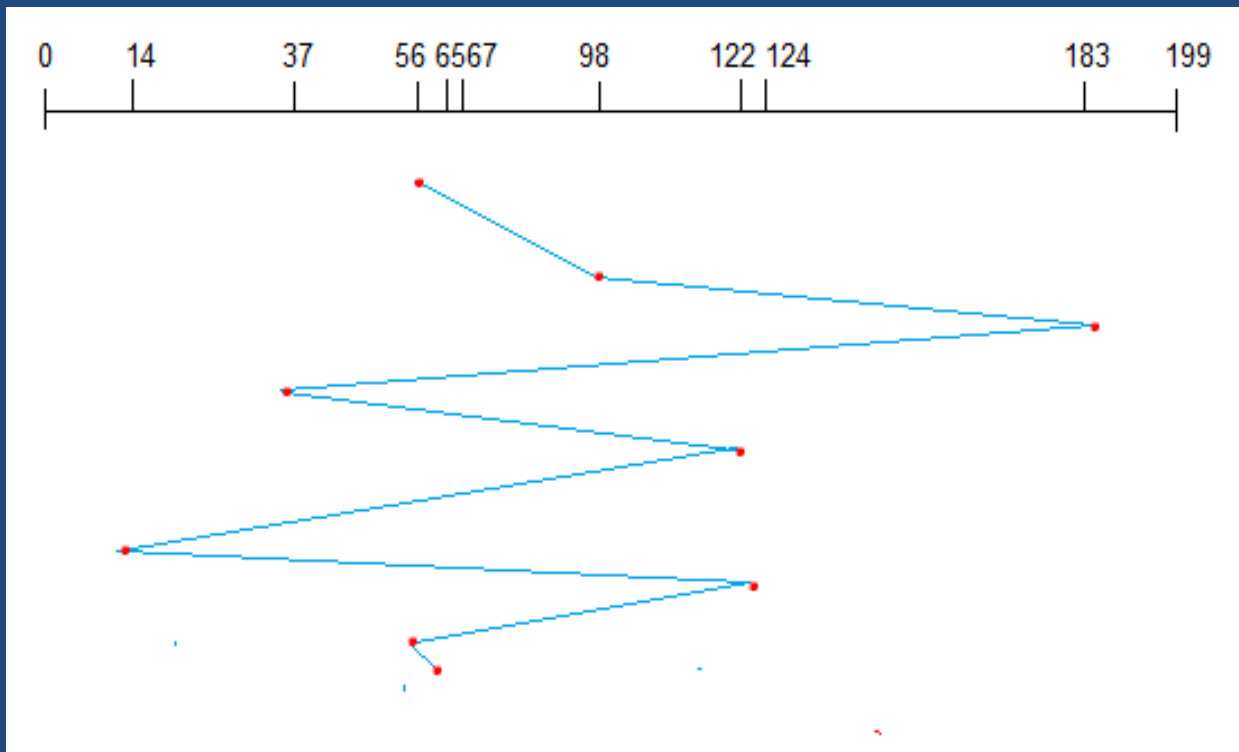
- **The speed of the disk is measured as two parts:**
- **Transfer rate:** This is the rate at which the data moves from disk to the computer.
- **Random access time:** It is the sum of the seek time and rotational latency.
- ❖ **Seek time** is the time taken by the arm to move to the required track.
- ❖ **Rotational latency** is defined as the time taken by the arm to reach the required sector in the track.
- ❖ Even though the disk is arranged as sectors and tracks physically, the data is logically arranged and addressed as an array of blocks of fixed size.
- ❖ The size of a block can be **512** or **1024** bytes.
- ❖ Each logical block is mapped with a sector on the disk, sequentially. In this way, each sector in the disk will have a logical address.

Disk Scheduling Algorithms:

- ❖ On a typical multiprogramming system, there will usually be multiple disk access requests at any point of time.
- ❖ So those requests must be scheduled to achieve good efficiency.
- ❖ Disk scheduling is similar to process scheduling.
- ❖ Some of the disk scheduling algorithms is described below.

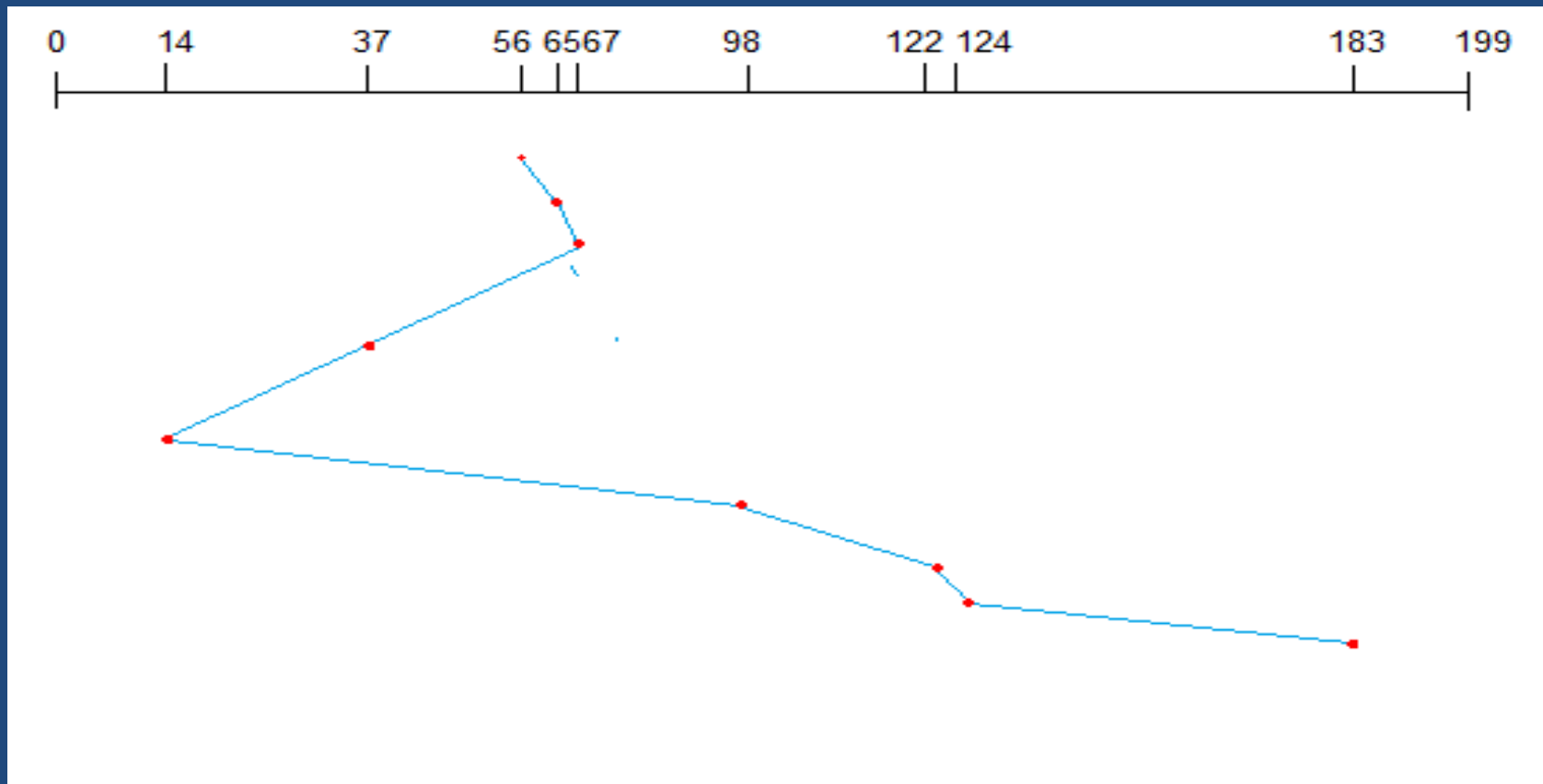
First Come First Serve

- ✓ This algorithm performs requests in the same order asked by the system.
- ✓ Let's take an example where the queue has the following requests with cylinder numbers as follows:
 - **98, 183, 37, 122, 14, 124, 65, 67**
 - Assume the head is initially at cylinder **56**. The head moves in the given order in the queue i.e., **56→98→183→...→67**.



Shortest Seek Time First (SSTF)

- Here the position which is closest to the current head position is chosen first.
- Consider the previous example where disk queue looks like,
 - **98, 183, 37, 122, 14, 124, 65, 67**
 - Assume the head is initially at cylinder 56. The next closest cylinder to 56 is 65, and then the next nearest one is 67, then 37, 14, so on.



SCAN algorithm

- This algorithm is also called the elevator algorithm because of its behavior.
- Here, first the head moves in a direction (say backward) and covers all the requests in the path.
- Then it moves in the opposite direction and covers the remaining requests in the path.
- This behavior is similar to that of an elevator.
- Let's take the previous example,
 - **98, 183, 37, 122, 14, 124, 65, 67**
 - Assume the head is initially at cylinder 56. The head moves in backward direction and accesses 37 and 14. Then it goes in the opposite direction and accesses the cylinders as they come in the path.

